자동매매 코드

📁 **파일 : main.py**

main.py는 자동매매 프로그램의 **실행을 담당하며**, 모든 하위 모듈을 순차적으로 초기화하고 **실시간 자동매매 루프를 시작**하는 역할을 한다.

동작 과정 및 순서:

1. 설정파일 로드(info.yaml) + 토큰 발급

```
# 1) 설정/토큰 로드
cfg = load_cfg()
token = get_token(cfg)
```

2. 후보 종목 선별(candidates.json)

```
candidates = load_or_create_candidates(cfg, token, limit=959, top_n=20)
```

3. 포지션 상태 복원 및 초기화

```
prev_state = load_positions()
state = initialize_state(candidates, buy_qty=BUY_QTY, prev_state=prev_state)
```

4. 실행 컨텍스트(ctx) 구성

```
# 3) 공유 컨텍스트 구성
ctx = {
    "cfg": cfg,
    "token": token,
    "market_env": "mock" if cfg.get("env","paper") in ("mock","paper") else "real",
    "take_profit": TAKE_PROFIT,
    "stop_loss": STOP_LOSS,
    "buy_qty": BUY_QTY,
    "order_cooldown": 1.0,
    "last_order_ts": {},  # code → last ts
    "state": state,
    "running": True,
}
```

5. 실시간 websocket 수신 + 매매로직 스레드 실행 + 장마감 자동 청산 루프 병행 실행

```
# 4) 실행 (WS + 장마감 청산)
t_ws = threading.Thread(target=start_ws_loop, args=(ctx,), daemon=True)
t_mc = threading.Thread(target=market_close_guard_loop, args=(ctx,), daemon=True)
t_ws.start()
t_mc.start()
```

6. 사용자 Ctrl+C 입력 시 종료 신호 전파 → 전체 루프 정상 종료

📁 **파일 : auth.py**

auth.py는 한국투자증권 Open API를 사용하기 위해 필요한 **인증(Authorization)** 절차를 수행하는 모듈이다. 다음 세 가지 기능을 제공한다:

1. info.yaml 설정파일을 로드 (load_cfg)
2. REST API 요청을 위한 액세스 토큰 발급 (get_token)
3. WebSocket 통신용 승인키 발급 (get_approval)

auth.py 함수별 설명

- load_cfg(path="../config/info.yaml")

역할:
프로젝트 설정파일(info.yaml)을 읽어 Python 딕셔너리 형태로 반환한다. info.yaml에는 사용자 키, 환경(paper/real), 도메인 주소 등의 주요 정보가 저장되어 있다.

입력값:

- path: YAML 파일 경로 (기본값 "../config/info.yaml")

반환값:

- 딕셔너리(cfg)

- get_token(cfg)

역할:
REST API를 호출하기 위한 **Access Token(액세스 토큰)**을 발급받는다.
이 토큰은 API 요청 시 Authorization: Bearer <token> 헤더에 포함되어야 한다.

입력값:

- cfg: load_cfg()를 통해 불러온 설정 딕셔너리

반환값:

- 문자열 형태의 Access Token

- get_approval(cfg)

역할:
실시간 WebSocket 통신을 위한 **승인키(Approval Key)** 를 발급받는다.
이 키는 체결가 수신(H0STCNT0) 등 WebSocket 구독 요청 시 필요하다.

입력값:

- cfg: 설정파일(info.yaml)을 로드한 딕셔너리

반환값:

- 문자열 형태의 Approval Key

📁 파일 : ohlcv.py

ohlcv.py는 한국투자증권 Open API를 이용해 **특정 종목의 최근 일별 시세 데이터(OHLCV)** 를 조회하는 모듈이다.

ohlcv.py 함수별 설명

- fetch_daily_ohlcv(cfg, token, code)

역할:
특정 종목의 최근 21일치 일봉 데이터를 REST API를 통해 수집한다. auth.py에서 발급받은 Access Token을 사용해 인증을 진행하며, 결과로 일자별 시세 정보(output2)를 리스트 형태로 반환한다.

입력값:

- cfg: auth.load_cfg()를 통해 불러온 설정 딕셔너리
  (API 키, 환경(paper/real), 도메인 주소 등 포함)
- token: auth.get_token(cfg)에서 발급받은 Access Token
- code: 조회할 종목 코드 (예: "005930" → 삼성전자)

반환값:

output2 필드에서 추출한 최근 21일치 일봉 데이터를 담은 리스트(list[dict])를 반환한다.

📁 파일 : order.py

order.py는 **한국투자증권 Open API의 주식 주문 기능**을 담당하는 모듈이다. REST API를 통해 실제 혹은 모의투자 환경에서 매수·매도 주문을 전송하고, 주문 결과를 로그 파일(core/data/order_log.txt)

에 기록한다.

order.py 함수별 설명

- _get_log_path()

역할:
주문 로그 파일(order_log.txt)의 절대 경로를 반환한다. 폴더(core/data/)가 존재하지 않을 경우 자동으로 생성한다.

입력값:

- 없음

반환값:

- str: 로그 파일의 절대 경로 (예: C:/project/core/data/order_log.txt)

- log_order_result(action, code, name, qty, price, res)

역할:
주문 결과를 로그 파일(order_log.txt)에 기록한다. 매수/매도 구분, 종목명, 수량, 가격, 결과 코드(rt_cd), 메시지(msg1), 주문번호(ODNO) 등의 정보를 저장한다.

입력값:

이름	자료형	설명
`action`	str	주문 유형 ("매수" 또는 "매도")
`code`	str	종목 코드 (예: `"005930"`)
`name`	str	종목명 (예: `"삼성전자"`)
`qty`	int	주문 수량
`price`	int	주문 단가 (0이면 시장가 주문으로 간주)
`res`	dict	주문 요청 결과(JSON 응답 객체)

반환값:

- 없음 (파일 기록 후 종료)

- send_order(cfg, token, code, name, qty, price, side="buy", market="mock")

역할:
한국투자증권 Open API의 "국내주식주문(현금)" API를 호출하여 **매수·매도 주문을 수행**한다. 주문 성공/실패 여부는 log_order_result()를 통해 로그로 남긴다.

입력값:

이름	자료형	설명
`cfg`	dict	`auth.load_cfg()` 를 통해 불러온 설정 딕셔너리
`token`	str	`auth.get_token(cfg)` 에서 발급받은 Access Token
`code`	str	종목 코드 (예: `"005930"`)
`name`	str	종목명 (예: `"삼성전자"`)
`qty`	int	주문 수량
`price`	int	주문 단가 (시장가일 경우 0)
`side`	str	`"buy"` 또는 `"sell"`
`market`	str	`"mock"` (모의투자) 또는 `"real"` (실거래)

반환값:

- dict 형태의 주문 응답 데이터

📁 파일 : screener.py

screener.py는 **KOSPI 종목 리스트 중 매매 후보 종목을 자동으로 선별**하는 모듈이다.
한국투자증권 Open API를 통해 각 종목의 최근 일봉 데이터를 받아, 변동성과 거래량을 기준으로 필터링을 수행한다. 이 모듈은 자동매매 시스템의 "시작 단계"로, **어떤 종목을 매매할지 결정하는 핵심 전처리 역할**을 한다.

screener.py 함수별 설명

- screen_candidates(cfg, token, stock_list, min_candidates=5)

역할:
주어진 주식 종목 리스트에서 조건1(변동성 감소) 과 조건2(거래량 감소) 를 모두 만족하는 종목을 필터링하여 매매 후보군(candidates)을 추출한다. 만약 필터링 결과가 너무 적을 경우, 조건2만 적용하여 다시 검색을 수행한다.

입력값:

이름	자료형	설명
`cfg`	dict	`auth.load_cfg()` 로 불러온 설정 딕셔너리
`token`	str	`auth.get_token(cfg)` 에서 발급받은 Access Token
`stock_list`	list[dict]	`get_kospi_stock_list_from_csv()` 로 불러온 KOSPI 종목 리스트 (각 원소: `{ "code": ..., "name": ... }`)
`min_candidates`	int	최소 후보 개수 (기본값: 5)

반환값:

- 자료형: list[dict]
- 필터링 조건을 만족한 종목들의 정보를 담은 종목 리스트.

📁 파일 : candidate_manager.py

candidate_manager.py는 **매매 후보 종목(candidates)** 데이터를 관리하는 모듈이다.
이 모듈은 candidates.json 파일을 불러오거나(load), 없을 경우 screener.py를 이용해 새로 생성(create)하는 기능을 담당한다.

candidate_manager.py 함수별 설명

- load_or_create_candidates(cfg, token, limit=959, top_n=20)

역할:
후보 종목(candidates.json)을 로드하거나 새로 생성한다. 이미 저장된 후보 파일이 존재하면 그대로 불러오고, 없을 경우 screen_candidates()를 호출해 새로운 후보를 만들어 저장한다. 후보종목군이 top_n개를 넘어간다면 상위 top_n개의 종목만 선택하여 반환한다.

입력값:

이름	자료형	설명
`cfg`	dict	설정 정보 (`auth.load_cfg()` 반환값)
`token`	str	API 인증 토큰 (`auth.get_token(cfg)` 에서 발급)
`limit`	int	전체 KOSPI 종목 중 앞에서 몇 개만 검사할지 (기본값: 959)
`top_n`	int	최종적으로 감시할 후보 종목 개수 (기본값: 20)

반환값:

- 자료형**:** list[dict]
- 감시 대상 후보 종목의 정보를 담은 리스트를 반환한다.

📁 파일 : kospy_list.py

kospy_list.py는 **KOSPI 종목 정보를 CSV 파일(kospi_data.csv)에서 불러오는 모듈**이다. 자동매매 프로그램의 초기 단계에서, 매매 대상이 될 수 있는 전체 KOSPI 종목 리스트를 로드하는 역할을 수행한다.

kospy_list.py 함수별 설명

- get_kospi_stock_list_from_csv()

역할:
core/data/kospi_data.csv 파일을 읽어 종목 코드(code) 와 종목명(name) 정보를 추출한다

입력값:

- 없음

반환값:

- 자료형: list[dict]
- KOSPI 종목 정보를 담은 리스트를 반환한다.

📁 파일 : state_manager.py

state_manager.py는 자동매매 시스템의 포지션(positions) 데이터를 관리하는 모듈이다. 매매 진행 중인 각 종목의 상태(매수/매도 여부, 목표가, 수량 등)를 positions.json 파일에 저장하고 불러오는 역할을 수행한다. 프로그램이 재시작되더라도 이전 포지션 상태를 복원할 수 있도록 유지 관리하는 기능을 담당한다.

state_manager.py 함수별 설명

- _get_positions_path()

역할:
포지션 상태 파일(positions.json)의 절대 경로를 반환한다. 폴더(core/data/)가 존재하지 않을 경우 자동으로 생성한다.

입력값:

- 없음

반환값:

- str: core/data/positions.json 파일의 절대 경로

- save_positions(state)

역할:
종목의 포지션 상태(state 딕셔너리)를 core/data/positions.json 파일에 저장한다. 자동매매 중 새 매수·매도 포지션이 생기면 이 함수가 호출된다.

입력값:

- state: 종목의 포지션 상태를 나타내는 dict 자료형 변수

반환값:

- 없음

- load_positions()

역할:
이전에 저장된 포지션 상태(positions.json)를 불러온다. 프로그램이 끊기고 재시작되더라도 이전 상태를 복원한다.

입력값:

- 없음

반환값:

- dict: 종목별 포지션 상태 정보

- initialize_state(candidates, buy_qty=1, prev_state=None)

역할:
후보 종목 리스트(candidates.json)를 기반으로 해당 종목군의 포지션 상태(state)를 초기화한다.

입력값:

이름	자료형	설명
`candidates`	list[dict]	매매 후보 종목 리스트 (code, name, target_price 등 포함)
`buy_qty`	int	각 종목별 기본 매수 수량 (기본값 1)
`prev_state`	dict	이전 포지션 상태 (`load_positions()` 결과, 기본값 None)

반환값:

- **자료형:** dict
- 초기화된 전체 포지션 상태를 반환한다.

📁 파일 : trade_logic.py

trade_logic.py는 자동매매 시스템의 **핵심 매매 판단 로직**을 담당하는 모듈이다. 실시간 체결 데이터(틱 데이터)를 기반으로 **매수 조건, 익절(이익 실현), 손절(손실 제한) 조건을 판단**하고, 해당 조건을 만족할 때 한국투자증권 API를 통해 주문을 실행한다. 또한 장마감 시점(15:20)에 **미청산 포지션을 자동 청산**하는 기능(market_close_guard_loop)을포함한다.

trade_logic.py 함수별 설명

- handle_tick(ctx, code, price)

역할:
실시간 체결가(틱 데이터)가 들어올 때마다 호출되어, 해당 종목이 매수·익절·손절 조건을 만족하는지 판단하고, 조건 충족 시 자동으로 주문(send_order)을 실행한다.

입력값:

이름	자료형	설명
`ctx`	dict	프로그램 실행 컨텍스트(환경 변수 및 상태 포함)
`code`	str	현재 체결 데이터가 들어온 종목 코드
`price`	float	현재 주가 (실시간 체결가)

반환값:

- 없음 (주문 실행 및 상태 저장 후 종료)

- market_close_guard_loop(ctx)

역할:
매일 장 마감 시각(15:20)에 **보유 중인 미청산 포지션을 자동으로 청산**한다.

입력값:

- ctx 변수

반환값:

- 없음

※ ctx(context)는 자동매매 시스템의 **전체 실행 상태를 담는 딕셔너리(dict)** 로, 각 모듈(trade_logic.py, websocket_loop.py, state_manager.py, order.py 등) 간에 필요한 설정, 상태, 매매 파라미터를 **공유하는 전역 컨텍스트 변수**이다.

📁 파일 : websocket_loop.py

websocket_loop.py는 한국투자증권 Open API의 **WebSocket 실시간 체결 데이터**를 수신받고, 각 종목의 가격 변동을 감지하여 trade_logic.py의 **매수·익절·손절 판단 로직(handle_tick)** 을 호출하는 모듈이다. 이 모듈은 **자동 재연결 기능**, **종목별 실시간 구독**, **메시지 파싱 및 예외 처리**를 포함하며, 자동매매 시스템의 실시간 동작 루프의 중심 역할을 한다.

websocket_loop.py 함수별 설명

- start_ws_loop(ctx)

역할:
WebSocket 연결을 유지하며 실시간 체결 데이터(H0STCNT0)를 받아 처리하는 메인 루프 함수.

입력값:

- ctx 변수

내부 동작 구조: start_ws_loop() 내부에는 WebSocket을 처리하는 4개의 서브 함수가 정의되어 있다.

1. **on_open(ws)**

역할: WebSocket 연결이 성공하면 각 종목의 체결 데이터(H0STCNT0)를 연결한다.

2. **on_message(ws, message)**

역할:
서버로부터 전달되는 실시간 메시지를 수신하고, handle_tick()으로 전달한다.

3. **on_error(ws, error)**

역할: WebSocket 통신 중 오류 발생 시 로그 출력

4. **on_close(ws, code, reason)**

역할: 서버 연결이 끊겼을 때 종료 이유를 출력

start_ws_loop() 본체에서는 while ctx["running"]: 루프를 유지하며 연결이 끊어질 때마다 5초 후 자동 재시도한다.

반환값:

- 없음
- 실시간 구독 루프이므로 반환값이 없으며, 내부적으로 체결 데이터 수신 → handle_tick() 호출 → 주문 실행 → 상태 저장 순으로 처리된다.